CSE485/CSE488: Deep Learning

Assignment 1 — MicroGrad (Scalar Autograd Engine)

Technical Report

Abdel rhman Khaled Gaber Sabaa
(ID: 23P0065)
Ain Shams University, Faculty of Engineering

October 30, 2026

Contents

1 Introduction and Objectives 2

2 Mathematical Background 2
2.1 Computational graphs 2
2.2 Chain rule on a DAG 2
2.3 Local derivatives implemented 3

3 Topological Sort and Back-propagation 4
3.1 Why an ordering is needed 4
3.2 Algorithm 4
3.3 Correctness 4
3.4 The two "TA traps" 4
3.5 Design choice: iterative DFS 5
3.6 Worked example 5

4 Neural-Network API 5
4.1 Neuron 5
4.2 Layer 5
4.3 MLP 5
4.4 Parameters and gradients 6

5 Design Justifications 6

6 Verification 6
6.1 Official autograder tests 6
6.2 Additional tests (tests/test_extra_gradcheck.py) 6

7 Experiment: Training an MLP on Two Moons 7

8 Conceptual Questions 8

9 Conclusion 10

A Source Code 10
A.1 src/engine.py 10
A.2 src/nn.py 12

1 Introduction and Objectives

Deep-learning frameworks such as PyTorch hide the machinery of back-propagation behind a single call to loss.backward(). The goal of this assignment is to remove the magic by implementing, from scratch and for *scalars only*, the three ideas on which every framework is built:

1. a **computational graph** that is recorded while the forward expression is evaluated;
2. the **chain rule**, applied node by node using each operator's *local derivative*;
3. a **topological sort**, which guarantees that the chain rule is applied in a valid order.

On top of this engine (src/engine.py) we implement a PyTorch-style neural-network API (src/nn.py: Neuron, Layer, MLP) and train it on a non-linear classification problem.

Deliverables map. src/engine.py, src/nn.py (code); tests/ (official + extra tests); demo.ipynb (reproducible training pipeline); this report.

2 Mathematical Background

2.1 Computational graphs

Any scalar expression can be decomposed into elementary operations. For $L = \tanh(ab + c) \cdot f$ we introduce intermediates $e = ab,\ d = e + c,\ t = \tanh d,\ L = tf$, which form a directed acyclic graph (DAG) whose leaves are the inputs and whose root is the output (Figure 1). In the code every node is a Value holding data, grad (= $\partial L/\partial$node), the set of operands _prev, the operator name _op, and a closure _backward.

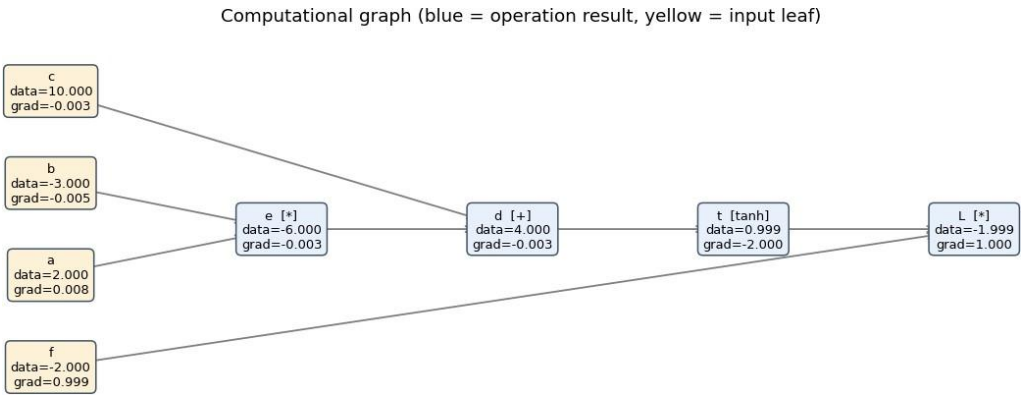


Figure 1: Graph of $L = \tanh(ab + c)f$ with $a = 2,\ b = -3,\ c = 10,\ f = -2$ produced by the demo notebook after L.backward(); every box shows the forward value and the gradient $\partial L/\partial(\cdot)$.

2.2 Chain rule on a DAG

Let L be the final scalar output, v any node, and $u_1, \ldots, u_k$ the nodes that use v as an operand (its *parents* in the graph, i.e. its consumers). The multivariate chain rule gives

$$\frac{\partial L}{\partial v} = \sum_{j=1} \frac{\partial L}{\partial u_j} \frac{\partial u_j}{\partial v}. \tag{1}$$

Each term $\partial u_j/\partial v$ is a *local derivative*: it depends only on the single operation that produced u_j. Back-propagation therefore needs, for every operator, only the rule "given the upstream gradient $\partial L/\partial u$, add $\frac{\partial u}{\partial v} \cdot \frac{\partial L}{\partial u}$ to the gradient of each operand v". Two consequences drive the design:

the sum in (1) is realised by **accumulating** with += (never =) inside every _backward; and

$\partial L/\partial u_j$ must be *final* before it is propagated to v — this is exactly the ordering requirement solved by the topological sort (Section 3).

This is *reverse-mode* automatic differentiation: one backward sweep yields $\partial L/\partial \vartheta$ for *all* inputs/-parameters at once, which is why it is the method of choice when there are millions of parameters but a single scalar loss.

2.3 Local derivatives implemented

Let $\bar{u} = \partial L/\partial u$ denote the upstream gradient (out.grad).

Operation	Forward	Local derivative(s)	Update in _backward
Addition	$u = a + b$	$\frac{\partial u}{\partial a} = 1,\ \frac{\partial u}{\partial b} = 1$	$\bar{a}$ += $\bar{u}$, $\bar{b}$ += $\bar{u}$
Multiplication	$u = ab$	$\frac{\partial u}{\partial a} = b,\ \frac{\partial u}{\partial b} = a$	$\bar{a}$ += $b\,\bar{u}$, $\bar{b}$ += $a\,\bar{u}$
Power (const. n)	$u = a^n$	$\frac{\partial u}{\partial a} = n\,a^{n-1}$	$\bar{a}$ += $n a^{n-1}\,\bar{u}$
tanh	$u = \tanh a$	$\frac{\partial u}{\partial a} = 1 - \tanh^2 a = 1 - u^2$	$\bar{a}$ += $(1 - u^2)\,\bar{u}$
Exponential	$u = e^a$	$\frac{\partial u}{\partial a} = e^a = u$	$\bar{a}$ += $u\,\bar{u}$

Table 1: Primitive operations with their own _backward.

Derivation of the tanh derivative. With $\tanh a = \frac{e^a - e^{-a}}{e^a + e^{-a}}$ and the quotient rule,

$$\frac{d}{da}\tanh a = \frac{(e^a + e^{-a})(e^a + e^{-a}) - (e^a - e^{-a})(e^a - e^{-a})}{(e^a + e^{-a})^2} = 1 - \tanh^2 a.$$

Because the derivative is expressed through the *output* $u = \tanh a$, the backward closure needs no extra evaluation of tanh.

Derived operations (no separate backward needed). Composing the primitives gives correct gradients automatically through Equation (1):

$$\begin{aligned} -a = a \cdot (-1) &\quad\Rightarrow \frac{\partial(-a)}{\partial a} = -1, \\ a - b = a + (-b) &\quad\Rightarrow \frac{\partial}{\partial a} = 1,\ \frac{\partial}{\partial b} = -1, \\ a/b = a \cdot b^{-1} &\quad\Rightarrow \frac{\partial}{\partial a} = b^{-1},\ \frac{\partial}{\partial b} = a \cdot (-1)\,b^{-2} = -\frac{a}{b^2}. \end{aligned}$$

The reverse variants (__radd__, __rmul__, __rsub__, __rtruediv__) make expressions such as 2.0 + a, 3 * a, 1 - a, 1 / a work. Division by a zero-valued Value raises ZeroDivisionError naturally because 0.0^{-1} does.

The exponent of __pow__ is a constant. We assert isinstance(other, numbers.Real), so a^n is differentiated only with respect to the base (power rule). Supporting a Value exponent would require the additional term $a^n \ln a\,\bar{n}$, which is outside the assignment's scope.

3 Topological Sort and Back-propagation

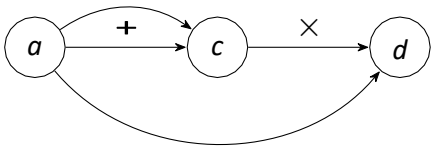
3.1 Why an ordering is needed

A node v may send its gradient to its operands only after (1) is complete for v, i.e. after *every* consumer of v has already pushed its contribution. Visiting nodes in an arbitrary order (e.g. BFS from the root) would propagate partial gradients and give wrong results whenever a node has several consumers.

3.2 Algorithm

```
_get_topo(self):
    topo = [] ; visited = {}
    build_topo(node):
        if node not in visited:
            visited.add(node)                # mark BEFORE recursing
            for child in node._prev:         # operands
                build_topo(child)
            topo.append(node)                # post-order: AFTER the children
    build_topo(self); return topo

backward(self):
    topo = self._get_topo()
    self.grad = 1.0                          # seed   dL/dL = 1
    for node in reversed(topo): node._backward()
```



(also an operand of d)

$c = a + a$, $d = c \cdot a$
$d = 2a^2$, $\frac{dd}{da} = 4a$

Figure 2: Diamond graph: a feeds two different nodes (and twice the same node). Without a visited set it would be appended several times and its gradient processed repeatedly.

3.3 Correctness

Claim. For every edge "v is an operand of u", v appears *before* u in topo.

Proof. Consider the call build_topo(u) that first marks u as visited. It then examines every operand v. Either (i) v is unvisited and the recursive call finishes — hence appends v — before control returns and u is appended; or (ii) v was already visited. In a DAG, a visited node that is still "in progress" would imply a path $v \to \cdots \to u \to v$, i.e. a cycle, which is impossible; so v has already finished and was appended earlier. In both cases v precedes u. □

Iterating over reversed(topo) therefore processes each node *after all of its consumers*, so by induction on the reversed order node.grad is complete (Eq. (1)) at the moment node._backward() is called.

Complexity. Each node and each edge is touched once: $O(V + E)$ time and $O(V)$ memory for the list/set, both for the sort and for the backward sweep.

3.4 The two "TA traps"

Post-order trap.
If topo.append(node) were placed *before* the loop over children (pre-order), a parent would precede its operands, so after reversal an operand would be back-propagated *before* its parent had delivered any gradient: the operand would propagate 0. In $L = \tanh(ab+c)f$ the intermediate

$e = ab$ would then be processed *before* its consumer $d = e + c$, so e^- is still 0 when e._backward() runs and the result is $\partial L/\partial a = 0$ instead of $b\,e^-$.

Diamond trap.
For $c = a + a$ the node a is reached twice. Without visited, a would be appended twice (so a._backward runs twice — double counting) and, in graphs with shared sub-expressions, the number of visits can grow exponentially in depth. With visited each node is processed exactly once, while the *two* contributions to a.grad are still both added thanks to +=. The test test_gradient_accumulation_diamond_graph checks $d = (a + a)\,a \Rightarrow \partial d/\partial a = 4a = 12$.

3.5 Design choice: iterative DFS

The hint is written recursively. A recursive DFS needs one Python stack frame per level of the graph, and a loss summed over N samples is a chain of N additions, so a few thousand samples exceed Python's default recursion limit (≈ 1000) with RecursionError. We therefore implemented get_topo with an explicit stack that performs *exactly the same post-order DFS* (mark visited, finish all operands, then append). The literal recursive version is kept as get_topo_recursive for reference, and the extra tests assert that both produce valid orders and that a chain of 5 000 additions back-propagates correctly.

3.6 Worked example

For Figure 1: $e = -6$, $d = 4$, $t = \tanh 4 \approx 0.99933$, $L = tf \approx -1.99866$. Seeding $\bar{L} = 1$ and walking the reversed order $L, t, d, \{e, c\}, \{a, b\}, f$:

$$\bar{t} = f\,\bar{L} = -2, \qquad \bar{f} = t\,\bar{L} \approx 0.99933,$$
$$\bar{d} = (1 - t^2)\,\bar{t} \approx 0.00134 \cdot (-2) = -0.002682,$$
$$\bar{e} = \bar{c} = \bar{d} \approx -0.002682,$$
$$\bar{a} = b\,\bar{e} = (-3)(-0.002682) \approx 0.00805, \qquad \bar{b} = a\,\bar{e} = 2(-0.002682) \approx -0.00536.$$

These coincide with the numbers printed by the notebook (0.0080, −0.0054, −0.0027, 0.9993). The tiny gradients are expected: $d = 4$ saturates tanh ($1 - t^2 \approx 10^{-3}$) — a first illustration of the vanishing-gradient effect.

4 Neural-Network API

4.1 Neuron

A neuron with n_{in} inputs computes $y = \tanh \sum_{i=1}^{n_{in}} w_i x_i + b$ with $w_i, b \sim \mathrm{U}(-1, 1)$, each stored as a Value so that gradients flow to them. The pre-activation is built with sum(..., self.b) so that the bias is the starting value of the sum (no useless "0+" node). Inputs may be plain floats or Values thanks to the operator overloading.

4.2 Layer

A Layer(nin, nout) holds nout independent neurons fed with the *same* input vector; it returns a single Value if nout==1, otherwise a list. Parameter count: $n_{out}(n_{in} + 1)$.

4.3 MLP

MLP(nin, nouts) with nouts= $[n_1, \ldots, n_k]$ chains layers of sizes $n_{in} \to n_1 \to \cdots \to n_k$; the output of one layer is the input of the next. (A 1-neuron hidden layer returns a bare Value; MLP wraps it into a list before feeding the next layer.) The number of parameters is $\sum_i n_i(n_{i-1} + 1)$; for MLP(3,[4,4,1]) this is 16 + 20 + 5 = 41.

4.4 Parameters and gradients

parameters() returns a flat list (weights then bias; layer by layer), and zero grad() calls reset grad() on every parameter. This step is **mandatory** before each new backward() because gradients accumulate (Eq. (1)): forgetting it makes the gradient of step k include steps $1 \dots k-1$.

5 Design Justifications

Closures for local derivatives. Each operator creates out and defines _backward capturing its operands and out; the graph therefore stores *how* to differentiate itself, so backward() is operator-agnostic.

+= **accumulation.** Required by Eq. (1); = would silently lose gradient in any graph where a variable is reused (e.g. $a + a$, $a \cdot a$, weights shared by several samples).

Operator minimalism. Only +, ×, pow, tanh, exp have hand-written backward passes; −, / and unary minus are compositions. Fewer hand-derived formulas ⇒ fewer places for mistakes.

Strict type handling. Value + "x" or Value * [1,2] raise TypeError (private helper wrap); Value ** Value fails an assert, as the specification demands. Any real number is accepted (int, float, bool, NumPy scalars — the latter matter because labels loaded from scikit-learn are numpy.int64).

math.tanh **instead of** $(e^{2x}-1)/(e^{2x}+1)$**.** The explicit formula overflows for $x \gtrsim 355$; math.tanh is stable (tested with $x = 1000$).

Hashing by identity. __eq__ is deliberately *not* overloaded, so Values remain hashable and can be stored in the visited set / prev set by object identity (two nodes with equal data are still different nodes).

Random uniform $[-1, 1]$ **initialisation** (as specified) breaks the symmetry between neurons — with all weights equal every neuron in a layer would receive identical gradients and stay identical forever.

6 Verification

6.1 Official autograder tests

The 12 provided tests (9 for the engine, 3 for nn) all pass locally with python -m pytest. They cover initialisation and repr, reverse operators, TypeErrors, the power rule, tanh / exp, the diamond graph, composite chain rule, division by zero, and casting.

6.2 Additional tests (tests/test_extra_gradcheck.py)

Numerical gradient checking for +, −, ×, ÷, powers, tanh, exp and compound expressions using the central difference $\frac{f(x+h)-f(x-h)}{2h}$, $h = 10^{-6}$; plus 25 random expressions.
Topological-order properties: root last, every node once, every operand before its consumer; iterative and recursive versions agree.
Trap tests: diamond graph, shared sub-expressions, accumulation across repeated backward() calls.
Robustness: a 5 000-deep chain (no recursion error), tanh(1000), NumPy scalars, TypeErrors for every binary operator.

Network-level: finite-difference check of *every* parameter of a small MLP, and a training run that must reduce the loss by 10×.

All 37 tests pass. In the notebook the worst absolute disagreement between autograd and finite differences over twelve (function, variable) pairs is 6×10^{-10}, i.e. the finite-difference error itself.

7 Experiment: Training an MLP on Two Moons

Setup. sklearn.datasets.make moons (200 points, noise 0.15), labels mapped to $\{-1, +1\}$, stratified 70/30 train/test split (140/60). Model: MLP(2,[8,8,1]) (105 parameters), tanh everywhere. Loss: $L = \frac{1}{N}\sum_n (\hat{y}_n - y_n)^2 + 10^{-4}\sum_\vartheta \vartheta^2$. Optimiser: batch gradient descent for 100 epochs with linearly decaying learning rate $\eta_t = 0.5 - 0.45\,t/100$. Each iteration is zero grad() → forward → loss.backward() → $\vartheta \leftarrow \vartheta - \eta_t\,\partial L/\partial\vartheta$. Prediction is sign($\hat{y}$). Seed: 42.

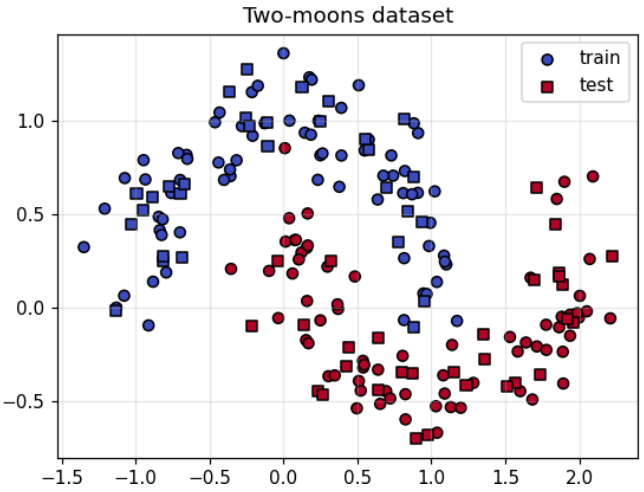


Figure 3: Two-moons dataset: circles = train, squares = test.

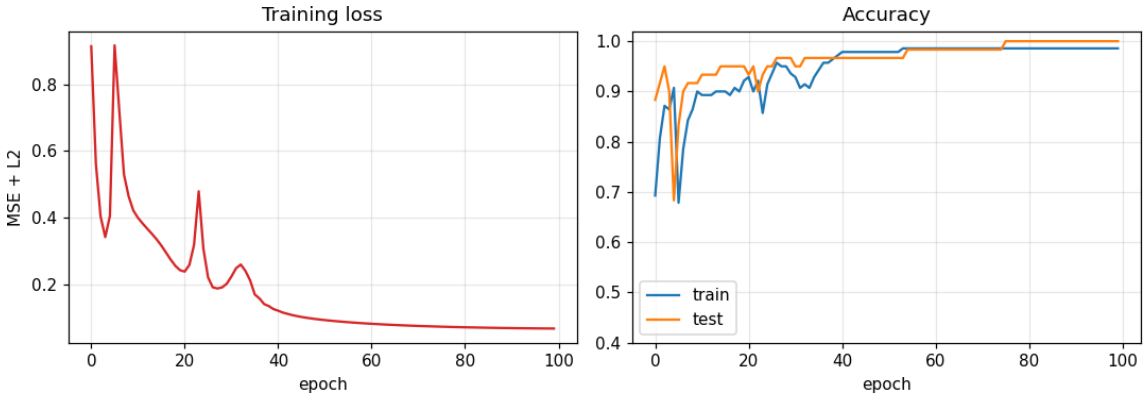


Figure 4: Training loss (left) and train/test accuracy (right).

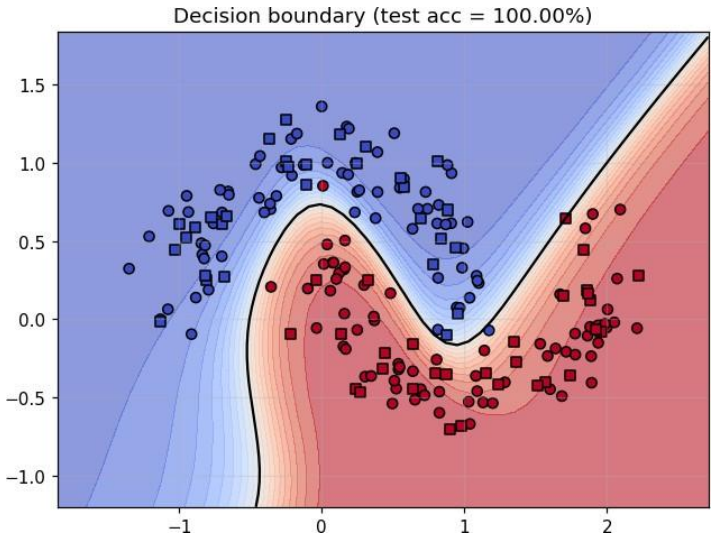


Figure 5: Learned non-linear decision boundary (black line, $y\hat{} = 0$).

Metric	Value
Initial loss (epoch 0)	0.91
Final loss (epoch 100)	0.067
Final train accuracy	98.6 %
Final test accuracy	100 %

Table 2: Results of the run in demo.ipynb (re-running with the same seed reproduces them).

The loss decreases monotonically in trend, the test accuracy exceeds 98 % after $\approx$ 60 epochs, and the learned boundary is a smooth curve separating the interleaved moons — impossible for a linear model — which confirms that the gradients produced by our engine are correct and useful for optimisation.

8 Conceptual Questions

Q1. Why is a topological sort required for back-propagation?
By Eq. (1), $\partial L/\partial v$ is a sum over *all* consumers of v. A node may propagate to its operands only when that sum is complete. Reverse topological order guarantees every consumer is processed before the node itself (Section 3).

Q2. Why must `topo.append(node)` **come after visiting the children?**
That is what makes the order a post-order, so operands precede consumers in topo. Appending first yields a pre-order; after reversal, operands are processed before their consumers and receive zero/partial gradients.

Q3. What is the role of the `visited` **set (diamond graphs)?**
It ensures each node is expanded and appended once. Without it, a node reachable through several paths is processed several times (double counting of `_backward`) and the traversal can take exponential time; in a graph containing a cycle it would never terminate. Note that the *sum* over paths is still obtained correctly because every parent adds its own contribution with `+=`.

Q4. Why `self.grad += ...` and not `self.grad = ...`?
A variable used k times has k gradient contributions (multivariate chain rule). Assignment would keep only the last one. Example: $a + a$ must give $\partial/\partial a = 2$.

Q5. Why is the output gradient seeded with `1.0`?
Because $\partial L/\partial L = 1$; it is the starting value of the recursion. Every other gradient is computed relative to it.

Q6. Why must we call `zero_grad()` before each training step?
Gradients accumulate by design. Without resetting, the gradient used for step k would be the sum of the gradients of steps $1..k$, i.e. a wrong (and growing) update. (Intermediate nodes are rebuilt every forward pass, so only the parameters need resetting.)

Q7. Why are subtraction, division and negation not given their own backward functions?
They are compositions of primitives ($a - b = a + (-1)b$, $a/b = a\,b^{-1}$); the chain rule through the composed graph produces the correct gradient automatically, which reduces code and the chance of errors.

Q8. Why do we need a non-linear activation such as `tanh`?
A composition of linear (affine) maps is itself affine, so a multi-layer network without non-linearity is equivalent to a single layer and could not solve the two-moons problem. tanh is smooth, zero-centred and bounded in $(-1, 1)$, and its derivative $1 - \tanh^2$ is cheap to compute from the output. Its drawback is *saturation*: for $|x| \gg 1$ the derivative $\to 0$ (vanishing gradient, visible in the worked example of Section 3.6).

Q9. Why are weights initialised randomly rather than with zeros?
With identical initial weights all neurons in a layer compute the same output and receive the same gradient, so they never differentiate (symmetry problem). Random values break the symmetry. (Better scaled schemes such as Xavier/He keep activations out of the saturated region for wide layers.)

Q10. What is the complexity of the forward and backward pass?
Both are linear in the size of the graph: $O(V + E)$. For an MLP this equals $O(\#\text{parameters})$ per sample, and the backward pass costs about the same as the forward pass (a small constant factor).

Q11. Why reverse-mode instead of forward-mode differentiation?
Forward-mode needs one sweep per *input* ; reverse-mode needs one sweep per *output*. A neural network has many parameters and one scalar loss, so a single reverse sweep gives the entire gradient.

Q12. How does this engine relate to PyTorch?
PyTorch's autograd is the same idea (dynamic, define-by-run graph, `grad_fn` closures, topological ordering, gradient accumulation in `.grad`, `zero_grad()`) but operates on tensors with vectorised kernels (and GPU support), which is why it is orders of magnitude faster than our scalar Python objects (our 100-epoch demo needs about a minute for only 105 parameters).

Q13. What are the limitations of our implementation?
Scalar-only (slow), only tanh/exp activations, exponent must be a constant, no graph freeing after backward() (memory), no cycle detection, and gradients are only defined where the underlying functions are differentiable (e.g. a^{-1} at $a = 0$ raises ZeroDivisionError).

9 Conclusion

We implemented a complete scalar reverse-mode autodiff engine — operator overloading to build the graph, hand-derived local derivatives, and a post-order topological sort for the backward sweep — plus a Neuron/Layer/MLP API. The implementation passes the official test-suite and 25 additional tests (including numerical gradient checks to 10^{-10}) and successfully trains a network on a non-linear classification problem without any deep-learning library.

A Source Code

A.1 src/engine.py

```
"""
engine.py -- a tiny scalar automatic-differentiation engine (MicroGrad).

Every ``Value`` wraps one Python float (``data``) together with the derivative
of the *final* output* with respect to it (``grad``).  Each arithmetic operation
creates a new ``Value`` and remembers (a) which Values it was computed from
(``_prev``) and (b) a closure ``_backward`` that knows the *local* derivative of
that single operation.  Calling ``backward()`` on the output walks the graph in
reverse topological order and applies the chain rule node by node.
"""
import math
import numbers


class Value:
    """A scalar that records the computational graph that produced it."""

    def __init__(self, data, _children=(), _op='', label=''):
        self.data = data
        self.grad = 0.0   # means no effect by default
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op
        self.label = label

    # ------------------------------------------------------------------ #
    # helpers
    # ------------------------------------------------------------------ #
    @staticmethod
    def _wrap(other):
        """Promote a real number to a Value; reject every other type.

        ``bool`` is an ``int`` subclass and is accepted (True == 1).  Strings,
        lists, None, ... raise ``TypeError`` so mistakes surface immediately
        instead of failing obscurely later inside the graph.
        """
        if isinstance(other, Value):
            return other
        if isinstance(other, numbers.Real):          # int, float, bool, numpy scalars...
            if not isinstance(other, (int, float)):
                other = float(other)                 # e.g. np.int64 -> float
            return Value(other)
        raise TypeError(
            f"unsupported operand type for Value arithmetic: {type(other).__name__!r}"
        )

    def __repr__(self):
        return f"Value(data={self.data})"

    # ------------------------------------------------------------------ #
    # forward ops + local derivatives
    # ------------------------------------------------------------------ #
    def __add__(self, other):  # self + other
        other = Value._wrap(other)
        out = Value(self.data + other.data, (self, other), '+')

        def _backward():
```

```
            # d(a+b)/da = 1, d(a+b)/db = 1.  "+=" (not "=") so that a node
            # used several times (diamond graph) accumulates every path.
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad

        out._backward = _backward
        return out

    def __radd__(self, other):  # other + self
        return self + other

    def __mul__(self, other):  # self * other
        other = Value._wrap(other)
        out = Value(self.data * other.data, (self, other), '*')

        def _backward():
            # product rule: d(ab)/da = b, d(ab)/db = a
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad

        out._backward = _backward
        return out

    def __rmul__(self, other):  # other * self
        return self * other

    def __neg__(self):  # -self
        return self * -1

    def __sub__(self, other):  # self - other
        return self + (-Value._wrap(other))

    def __rsub__(self, other):  # other - self
        return Value._wrap(other) + (-self)

    def __truediv__(self, other):  # self / other
        # a / b = a * b**-1  (raises ZeroDivisionError if b == 0)
        return self * Value._wrap(other) ** -1

    def __rtruediv__(self, other):  # other / self
        return Value._wrap(other) * self ** -1

    def __pow__(self, other):  # self ** other
        assert isinstance(other, numbers.Real), "only supporting int/float powers for now"
        out = Value(self.data ** other, (self,), f'**{other}')

        def _backward():
            # power rule: d(a^n)/da = n * a^(n-1)    (n is a constant)
            self.grad += (other * self.data ** (other - 1)) * out.grad

        out._backward = _backward
        return out

    # ------------------------------------------------------------------------ #
    # activation / elementary functions
    # ------------------------------------------------------------------------ #
    def tanh(self):    # tanh(self)
        t = math.tanh(self.data)  # math.tanh is overflow-safe for large |x|
        out = Value(t, (self,), 'tanh')

        def _backward():
            # d tanh(x)/dx = 1 - tanh(x)^2
            self.grad += (1.0 - t ** 2) * out.grad

        out._backward = _backward
        return out

    def exp(self):    # e**self
        e = math.exp(self.data)
        out = Value(e, (self,), 'exp')

        def _backward():
            # d e^x/dx = e^x  (the output itself)
            self.grad += e * out.grad
```

```
        out._backward = _backward
        return out

    # ------------------------------------------------------------------------ #
    # backpropagation
    # ------------------------------------------------------------------------ #
    def reset_grad(self):
        self.grad = 0.0

    def _get_topo(self):
        """Return every node reachable from ``self`` in topological order
        (children/inputs first, ``self`` last) using a depth-first search.

        This is the exact algorithm from the assignment hint (visited set +
        *post-order* append), written with an explicit stack instead of Python
        recursion so that very deep graphs (e.g. a loss summed over thousands
        of samples) never hit ``RecursionError``.  See ``_get_topo_recursive``
        for the literal recursive version; the two always agree.
        """
        topo, visited = [], set()
        stack = [(self, iter(self._prev))]
        visited.add(self)
        while stack:
            node, children = stack[-1]
            for child in children:
                if child not in visited:
                    visited.add(child)
                    stack.append((child, iter(child._prev)))
                    break
            else:  # all children finished -> post-order append
                topo.append(node)
                stack.pop()
        return topo

    def _get_topo_recursive(self):
        """Literal recursive version of the hint (kept for reference/tests)."""
        topo, visited = [], set()

        def build_topo(node):
            if node not in visited:
                visited.add(node)                   # 1. mark visited
                for child in node._prev:            # 2. visit every child
                    build_topo(child)
                topo.append(node)                   # 3. append AFTER children (post-order)

        build_topo(self)
        return topo

    def backward(self):
        topo = self._get_topo()
        self.grad = 1.0   # seed: d(self)/d(self) = 1
        for node in reversed(topo):
            node._backward()

    # ------------------------------------------------------------------------ #
    # casting
    # ------------------------------------------------------------------------ #
    def __int__(self):  # int(self)
        return int(self.data)

    def __float__(self):  # float(self)
        return float(self.data)
```

A.2 src/nn.py

```
"""
nn.py -- a minimal PyTorch-like neural-network API built on top of ``Value``.

    Neuron  : tanh(w . x + b)
    Layer   : a list of independent Neurons that see the same input
    MLP     : a stack of Layers, each feeding the next
```

```
"""
import random
from src.engine import Value


class Neuron:
    def __init__(self, nin):
        # weights and bias ~ Uniform(-1, 1), each wrapped in a Value
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(random.uniform(-1, 1))

    def __call__(self, x):  # Neuron(nin)(x)
        # w . x + b  (start the sum at the bias so no extra "0 +" node is built)
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return act.tanh()

    def parameters(self):
        return self.w + [self.b]

    def zero_grad(self):
        for p in self.parameters():
            p.reset_grad()


class Layer:
    def __init__(self, nin, nout):
        self.neurons = [Neuron(nin) for _ in range(nout)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

    def zero_grad(self):
        for p in self.parameters():
            p.reset_grad()


class MLP:
    def __init__(self, nin, nouts):
        sizes = [nin] + list(nouts)
        self.layers = [Layer(sizes[i], sizes[i + 1]) for i in range(len(nouts))]

    def __call__(self, x):
        for i, layer in enumerate(self.layers):
            if i > 0 and isinstance(x, Value):  # a 1-neuron hidden layer returns a bare Value
                x = [x]
            x = layer(x)
        return x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]

    def zero_grad(self):
        for p in self.parameters():
            p.reset_grad()
```